E & D Group — Database Design Spec

Online game live-ops analytics

IFT 530: Advanced Database Management Systems · Final Project · Ethan Oxley, Daniel Warren · Arizona State University

Overview

Our project builds the analytics database behind an online free-to-play game — the kind of system a live-ops team actually runs on. We instrument every player action: sessions, levels played, items used, ads watched, purchases, and the location the player was in when they did it. Players arrive through marketing campaigns, so acquisition cost rides along on the user record and we can answer whether a campaign ever paid for itself. Accounts sit on a plan — free, Plus, Premium — which lets us compare free versus paid *behavior* rather than just free versus paid revenue.

We landed on a star schema: six dimensions around three fact tables, plus one audit table. Geography is the piece we are most pleased with — a role-playing dimension referenced five times, including both a user's registered location and their last-seen location. That single choice gave us fraud detection almost for free.

DATABASE	`E_D_Group` on SQL Server 2025 Developer Edition
MODEL	Star schema — 6 dimensions, 3 facts, 1 audit table (10 total, the assignment cap)
INTEGRITY	15 foreign keys, 10 primary keys, 3 unique constraints, 32 check constraints, 7 defaults
WINDOW	30 days — 2026-09-26 through 2026-10-25, spanning two calendar months
VOLUME	10 rows per dimension, 20–50 per fact table (as the assignment requires); schema designed for 100,000 users

Dimensions

TABLE	ROWS	KEY COLUMNS	WHAT IT SLICES
`DimGeography`	10	`GeoID` PK, CountryCode, CountryName, Region, City, **Latitude**, **Longitude**, RiskTier	Role-playing — referenced 5× for registered, last-seen, billing, event and payment location
`DimUser`	10	`UserID` PK, Username, Email, SignupDate, **RegisteredGeoID**, **LastSeenGeoID**, LastSeenAt, AgeBand, PreferredDevice, LifecycleStage, AcqCampaignID, AcqCostUSD	The person — demographics, device, cohort, lifecycle stage, acquisition cost
`DimAccount`	10	`AccountID` PK, UserID, AccountNumber, PlanID, AccountStatus, OpenedDate, ClosedDate, IsAutoRenew, BillingGeoID	The billing entity, deliberately separate from the person

TABLE	ROWS	KEY COLUMNS	WHAT IT SLICES
DimPlan	10	PlanID PK, PlanName, PriceUSD, BillingPeriod, **IsPaid**, TrialDays, AdsEnabled, EffectiveDate	Free vs paid tiers — this is the audited lookup table
DimProduct	10	ProductID PK, ProductName, ProductCategory, ListPriceUSD, IsActive, ReleaseDate	Store catalog, kept apart so a price change does not rewrite history
DimCampaign	10	CampaignID PK, CampaignName, Channel, CostModel, StartDate, EndDate, BudgetUSD	Marketing attribution and channel spend

Fact tables

TABLE	GRAIN	ROWS	CARRIES
FactUserEvent	One row per player action	48	SessionID *(degenerate dimension)*, EventType, EventTS, LevelNumber, Outcome, DeviceType, SourceSystem
FactPayment	One row per transaction	30	AmountUSD, PaymentType, PaymentMethod, GeoID, IsFlagged, SourceSystem
FactSubscription	One row per lifecycle change	23	EventType, **PrevPlanID**, MRRDeltaUSD, TermMonths, CancelReason, SourceSystem

DimPlanAudit is the tenth table, written only by the trigger. It has *no* foreign key back to DimPlan on purpose: a DELETE audit row has to outlive the row it describes, and a foreign key would either block the delete or cascade the history away.

Relationships

```
Create order (drop order is its exact reverse):

  DimGeography, DimPlan, DimProduct, DimCampaign   (no outbound FKs)
    -> DimUser
         -> DimAccount
              -> FactUserEvent, FactPayment, FactSubscription

  DimPlanAudit                       (standalone, written by the trigger)
```

Nothing closes a loop, so the script is re-runnable. Dimension keys are assigned explicitly so the populate section can reference them deterministically; fact keys are IDENTITY, because at the volume this schema is designed for nothing should be assigning keys by hand.

Programmable objects

OBJECT	TYPE	WHAT IT DOES
vwDailyActiveUsers	View	DAU across the 30-day window, split free vs paid
vwFreeVsPaidValue	View	Engagement and lifetime value by plan
vwSuspiciousTransactions	View	Impossible-travel and high-risk geography — 5 of 30 payments flagged
fnGeoDistanceKm	Scalar UDF	Haversine great-circle distance between two GeoIDs
spMonthlyActiveUsers	Procedure	MAU, revenue and ARPU for a month; THROW 50001/50002 on bad parameters
TR_DimPlan_Audit	Trigger	One AFTER INSERT, UPDATE, DELETE trigger covering all three operations
curLevelDifficulty	Cursor	GLOBAL cursor walking the levels, flags any level under a 50% win rate as a churn risk

Design rules we held to

- **Every coded column is constrained.** 32 check constraints — no free-text status, channel, device or event-type columns anywhere.
- **Indexes are for the target volume, not the sample.** Six non-key indexes on the two timestamp columns and the hot join columns. At 10 users the optimizer ignores them and scans every time; they exist because the schema is designed for 100,000.
- **Descriptive data lives in dimensions.** The fact tables carry integer keys, one timestamp and short coded columns; everything readable is reached by a join.
- **Prices are snapshotted, not referenced.** FactPayment.AmountUSD is what was actually charged, so a later catalog price change cannot rewrite revenue history.
- **Re-runnable.** Drops child-first, rebuilds parent-first; verified over three consecutive runs, 13 PASS / 0 FAIL each time.

> **Making ten rows say something.** Ten players over a short window will not demonstrate a reporting database by accident, so the data is laid out deliberately rather than drawn at random. The window crosses a month boundary so the monthly procedure returns two rows. Twelve sessions sit on nine days, three of them carrying more than one player, so daily active users moves between one and three instead of being flat. Nine accounts are open across both a free and a paid plan. Level five is lost three times in four. Three payments land in countries the account holder was nowhere near minutes earlier. Each of those is something a view, the procedure or the cursor depends on, and the generator asserts every one before writing a single INSERT.